6
Connecting the Dots

Throughout the book you have seen many examples of how to use Spring AI's APIs along with full-featured applications, like Shazam, which can be implemented using Spring AI and your choice of AI model. Now it's time to have some fun and play a game.

In this chapter, we put AI to the test and garner insight on how well different AI models can reason and perform under pressure in a competitive game of Connect 4. This is not a competition between humans and machine, but machine versus machine. We want to observe how well AI uses information passed in a prompt to make decisions about the next best possible move in the game to win!

In case you are not familiar with Connect 4, it is a simple game where you try to connect 4 discs of the same color in the same row, column or diagonally before your opponent. It is just you versus one other person, like in checkers or chess. The game can switch from offense to defense quickly if you are not careful. Sometimes you are planning several moves ahead to set your opponent up while other times you are scrambling to prevent your opponent from winning by countering their moves. You really have to pay attention. You can find more information on Connect 4 online at https://en.wikipedia.org/wiki/Connect_Four.

In this chapter, we will cover:

- Implementing Connect 4
- Prompts and Applied Features of Spring AI
- Explanations and AI Model Reasoning

Specifically, you will learn:

- How to use multiple AI models in your Spring AI applications
- Prompts and features of Spring AI required to implement Connect 4
- How well do AI models reason
- Whether reasoning models outperform non-reasoning models

We start by reviewing the components of the game's implementation.

Technical requirements

The Connect 4 application presented in this chapter is based on Java 17 and Spring Boot 3.5.x. The app uses Spring AI 1.0.x.

You must manually install JDK 17 or later. The project Maven POM declares and resolves Spring AI, Spring Boot and all required dependencies from Maven Central. You can find the source code for the Connect 4 application in the GitHub Repository:

https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/connect-four-app

This app prompts the user to select 2 AI models from a declared set of AI models that includes OpenAI, Ollama, MistralAI, and Google Vertex AI Gemini, to compete in a game of Connect 4.

You must set up an OpenAI account at https://platform.openai.com/signup and purchase credits to make API calls. After setting up your OpenAI account, create an API key and set the following Spring AI property in Spring Boot `application.properties`, like so:

```
spring.ai.openai.api-key=<your-openai-api-key>
```

To use Google Gemini, you must set up a Google Cloud account at https://console.cloud.google.com/freetrial/signup/tos and install the Google Cloud CLI from https://cloud.google.com/sdk/docs/install. You will use the Google CLI to authenticate with your Google Cloud account.

After your account is set up, you must create a new project and enable the Vertex AI Gemini service. You will need to enter a billing account in order to use GCP services.

Finally, after your project has been created, set the following Spring AI property to the ID of your new Google Cloud project in the Spring Boot `application.properties`, like so:

```
spring.ai.vertex.ai.gemini.projectId=<your-gcp-project-id>
```

To use MistralAI, you must set up a MistralAI account at https://auth.mistral.ai/ui/registration. You will then proceed to create an API key and set the following Spring AI property in Spring Boot `application.properties`, like so:

```
spring.ai.mistralai.api-key=<your-mistralai-api-key>
```

If you read any of the previous chapters, you probably already know how to set up Ollama. Download and install Ollama on your computer from https://ollama.com/download.

While the application uses Meta's Llama3.2 model by default, you can pull and use any of the available AI models at https://ollama.com/search. For instance, if you are interested in trying DeepSeek-R1, run:

```
$ ollama run deepseek-r1
```

Once the model has been pulled or run locally, it becomes available to use with Spring AI simply by setting the appropriate Spring AI property in the Spring Boot application.properties, like so:

```
spring.ai.ollama.chat.options.model=deepseek-r1:latest
```

For AI provider accounts that use an API key, be sure to keep it a secret. The README from the project GitHub repository will provide further instructions and guidance when using the app.

Finally, this example makes use of custom Spring AI extensions, which are covered in more detail in *Chapter 7*. While use of specific extensions is described in the context of the use case, please refer to *Chapter 7* for a complete description of available extensions and their functionality.

Implementing Connect 4

For simplicity the interface of the Connect4 application was implemented as a simple text-based, command-line interface (**CLI**).

The following output from the application on startup illustrates that the game is ready to be played:

```
$ mvnw --projects connect-four-app spring-boot:run
...
  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v3.5.6)
...
2025-10-26T18:43:36.525-07:00  INFO 60650 --- [Connect4 App] [           main] c.p.s.a.e.c.ConnectFourApplication       : Started ConnectFourApplicatio
```

Once the application is up and running, the user is presented with a selection of AI models to choose from that will play and compete in Connect 4.

```
Welcome to Connect4!
1. Ollama
2. OpenAI
3. Google Vertex AI Gemini
4. Mistral AI

Select player one: 1
Select player two: 3
Player 1 [Ollama] is playing [O]
Player 2 [Google Vertex AI Gemini] is playing [X]
```

Once the AI models (players) has been selected, a single AI model (player) is randomly selected to make the first move:

```
Current player is [Ollama] playing [O]
2025-10-26T18:44:27.905-07:00  INFO 60650 --- [Connect4 App] [           main] c.ConnectFourApplication$$SpringCGLIB$$0 : AI model explanation [?]
2025-10-26T18:44:27.906-07:00  INFO 60650 --- [Connect4 App] [           main] c.ConnectFourApplication$$SpringCGLIB$$0 : AI model decision durati
 A  B  C  D  E  F  G
 --- --- --- --- --- --- ---
|  |  |  |  |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  |  |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  |  |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  |  |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  |  |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  |  | O|  |  |
 --- --- --- --- --- --- ---
```

The AI models are instructed to state their move and provide an explanation for the move they chose. However, not all AI models are forthcoming and willing to share.

If an AI model does not provide a reason for its move, then a "?" will be displayed in place of the explanation. In addition to logging the move and explanation, the application also logs the amount of time it took for the AI model to make its decision.

The AI models take turns playing Connect4 until there is a winner, or there are no more available moves and the game results in a draw.

The last 2 moves of the current game appeared as follows:

```
Press <enter> to continue next play
Current player is [Google Vertex AI Gemini] playing [X]
2025-10-26T18:56:03.834-07:00  INFO 60650 --- [Connect4 App] [           main] c.ConnectFourApplication$$SpringCGLIB$$0 : AI model explanation [I
2025-10-26T18:56:03.835-07:00  INFO 60650 --- [Connect4 App] [           main] c.ConnectFourApplication$$SpringCGLIB$$0 : AI model decision durati
 A  B  C  D  E  F  G
 --- --- --- --- --- --- ---
|  |  |  |  |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  |  |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  |  |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  |  | O|  |  |
 --- --- --- --- --- --- ---
|  |  |  | X| O|  |  |
 --- --- --- --- --- --- ---
|  |  | X| X| O|  |  |
 --- --- --- --- --- --- ---
You will "win" on your next turn? Uhm.
Press <enter> to continue next play
Current player is [Ollama] playing [O]
2025-10-26T18:56:13.830-07:00  INFO 60650 --- [Connect4 App] [           main] c.ConnectFourApplication$$SpringCGLIB$$0 : AI model explanation [Pl
2025-10-26T18:56:13.831-07:00  INFO 60650 --- [Connect4 App] [           main] c.ConnectFourApplication$$SpringCGLIB$$0 : AI model decision duratio
 A  B  C  D  E  F  G
 --- --- --- --- --- --- ---
|  |  |  |  |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  |  |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  |  | O|  |  |
 --- --- --- --- --- --- ---
|  |  |  |  | O|  |  |
 --- --- --- --- --- --- ---
|  |  |  | X| O|  |  |
 --- --- --- --- --- --- ---
|  |  | X| X| O|  |  |
```

```
 --- --- --- --- --- --- ---
[Ollama] playing [O] wins!
```

Ollama using DeepSeek-R1 beat Google's Vertex AI Gemini 2.5 flash lite model!

In most cases, OpenAI's GPT-5 model consistently beats every other AI model over several rounds. We will come back to the results and this topic more in a bit.

Like previous examples and Spring AI applications presented in this book, Connect4 is built and bootstrapped as a Spring Boot application.

The main Java class used to bootstrap the Connect4 application is shown below:

```
@SpringBootApplication
@Profile(ConnectFourApplication.CONNECT_FOUR_PROFILE)
public class ConnectFourApplication
    extends AbstractConnectFourApplication {
  static final String CONNECT_FOUR_PROFILE = "connect4";
  public static void main(String[] args) {
    runSpringApplication(ConnectFourApplication.class, useProfiles(CONNECT_FOUR_PROFILE), args);
  }
  // ...
}
```

Reusable bits of functionality, such as prompting the AI models, are encapsulated in the `AbstractConnectFourApplication` class for convenience. For example, the following method from the abstract Java class (POJO) simplifies the use of Spring AI to prompt the AI model:

```
Play promptRealModel(String model,
    Map<String, Object> promptTemplateArguments,
    ChatClient chatClient) {

  BeanOutputConverter<Play> playConverter =
    new BeanOutputConverter<>(Play.class);

  JsonSanitizerOutputConverter<Play> jsonConverter =
    JsonSanitizerOutputConverter.from(playConverter);

  return chatClient.prompt()
    .system(systemPromptTemplate())
    .user(promptUserSpec ->
      promptUserSpec.text(userPromptTemplate())
        .params(promptTemplateArguments))
    .options(Utils.buildChatOptions(model))
    .call()
    .entity(jsonConverter);
}
```

The use of `ChatClient` to prompt the targeted AI model by name should be immediately familiar from *Chapter 2*. The method makes use of Spring AI's `BeanOutputConverter`, a `StructuredOutputConverter`, to convert JSON responses returned by the AI model into proper Connect4 application domain objects, like Play. However, this is not before the JSON is sanitized using a custom Spring AI `StructuredOutputConverter` to sanitize the JSON output returned in the AI model's response. Both the `abstract systemPromptTemplate()` and `userPromptTemplate()` methods simply allow both system and user prompts to be customizable by subclass.

Sanitization involves the use of Google's `JsonSanitizer` class to sanitize the JSON returned in the AI model's response and make sure the JSON data is well-formed. Otherwise, parsing errors can occur, as I learned the hard way. Despite explicit instructions, AI models are not always good at following them.

Both `JsonSanitizerOutputConverter` and `Utils.buildChatOptions(..)`, which returns nothing more than `ChatOptions.builder().model(modelName).build()`, merely simplify the implementation of the Connect4 application. Both classes come from custom Spring AI extensions presented in *Chapter 7*. Custom extensions are discussed in more detail in that chapter.

The configuration for the Spring Boot application is simple:

```
@SpringBootConfiguration
@EnableChatClient
static class ConnectFourConfiguration {
  @Bean
  ConnectFourBoardGame boardGame() {
    return new ConnectFourBoardGame();
  }
}
```

The declaration of the @EnableChatClient annotation from the Spring AI Extensions presented in **Chapter 7** enables the use of a Spring AI ChatClient to prompt the AI models.

The configured ChatClient is composed of multiple AI models declared on the application's classpath, which are treated as a single ChatModel using the *Composite* design pattern. The composition of AI models is encapsulated in a CompositeChatModel instance from Spring AI Extensions.

NOTE

In a nutshell, the @EnableChatClient annotation makes use of Spring Boot *auto-configuration* to automatically configure a ChatClient instance from multiple ChatModel objects declared on your Spring Boot application's classpath. It is similar to ChatClient.builder(chatModel).build(). However, instead of passing in a single ChatModel, the CompositeChatModel collects all configured ChatModels and is passed to the builder method instead, like so: builder(compositeChatModel). The CompositeChatModel allows the current ChatModel to be swapped at runtime.

Refer to the *Composite* design pattern (https://en.wikipedia.org/wiki/Composite_pattern) for more details.

TIP

You can learn more about the Spring AI Extensions in **Chapter 7**.

The configured AI models are declared in the project Maven POM and can be seen below:

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-mistral-ai</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-ollama</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-openai</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-vertex-ai-gemini</artifactId>
</dependency>
```

The dependency declarations declare Spring AI Model Starters for OpenAI, Ollama, MistralAI and Google Gemini.

The main logic for the Connect4 game is maintained in a configured (*Singleton*) bean instance of type ConnectFourGameBoard, which keeps track of players and the state of the game board for the duration of the game.

The main REPL (*Read, Evaluate, Print, Loop*) of the application is performed in the playGame(..) method. A partial implementation can be seen here:

```
@Bean
ApplicationRunner playGame(Environment environment, ChatClient chatClient, CompositeChatModel chatModel, ConnectFourBoardGame boardGame) {
  return args -> {
    print("%n%nWelcome to Connect4!%n%n");
    Scanner input = new Scanner(System.in);
    Players players = selectPlayers(input);
    Player currentPlayer = players.startingPlayer(SECURE_RANDOM, chatModel);
    while (boardGame.isPlayable()) {
```

```
    long timestamp = System.currentTimeMillis();
    Play play = promptModel(model, promptTemplateArguments, chatClient);
    Duration milliseconds = durationInMilliseconds(timestamp);
    PlayerAction playerAction = PlayerAction.by(currentPlayer)
     .played(play).in(milliseconds);
    playSafely(gameBoard -> {
     gameBoard.play(playerAction);
     gameBoard.printGameBoard();
    }).accept(boardGame);

    if (boardGame.isPlayable()) {
     currentPlayer = players.switchPlayer(chatModel);
     print("Press <enter> to continue next play ");
     waitOnUserInput(input);
    }
   }
   endGame(boardGame, players);
  };
 }
```

The `playGame(..)` method returns a Spring Boot `ApplicationRunner`, which is called on application startup by Spring Boot after the Spring `ApplicationContext` is initialized.

The AI models, acting as players, competing against each other in Connect4, are selected by the user in the `playGame(..)` method. A player is randomly chosen to play first. The AI model is prompted for its move, which gets recorded in the game board. The player switches and play continues until there is a winner.

The `ConnectFourGameBoard.isPlayable()` method returns `false` the moment a player wins or there are no more available moves to play. Play is paused between each move so users can observe the move made by an AI model and its explanation.

The output is colored by player to make it easier to review the plays. Player X is red. Player O is gold. The current move is presented in green to make it more apparent which was the last move played.

Again, the full state of the game is managed in a single instance of the `ConnectFourGameBoard` class. The logic in this class is not complicated and so I will not go over the class in detail here. However, the class uses several static inner classes to help model components of the game board and manage the game's state.

Along with static inner classes, several other key model types are used to represent a `Player` as well as a collection of `Players`. The `Player` type is defined as a Java record. The definition of `Player` is shown below:

```
public record Player(AiProvider provider, Disc disc) {
 // ...
}
```

The `AiProvider` interface is another Spring AI Extension that effectively represents the AI model using a more descriptive name, such as "Google Vertex AI Gemini."

The `Disc` Java enum represents the player on the game board, and is defined as:

```
public enum Disc {
  RED("X", AnsiColors.RED),
  GOLD("O", AnsiColors.YELLOW);
}
```

Let's jump into the next section and talk about the features of Spring AI used to implement the Connect4 game. I will also present the remaining application domain model types, which were essential in our use of Spring AI to implement Connect4.

Prompts and applied features of Spring AI

Spring AI's ChatClient interface is an essential component in prompting AI models and accessing additional features of Spring AI. Therefore, I will demonstrate both ChatClient and ChatModel's use in more detail below in addition to the prompts that were sent to the AI models.

But first, the `ChatClient` and `ChatModel` interfaces weren't the only two Spring AI features used in Connect4. Oftentimes, the use of a `ChatModel` is hidden behind the direct use of a `ChatClient`. Furthermore, when you use the `ChatClient` API, you typically make use of *Prompt Templates* as discussed in **Chapter 2**.

The Connect4 Spring AI application also uses `StructuredOutputConverters` presented in **Chapter 3** with `ChatClient` to format the generated response returned by the AI models. In total, we used the following features of Spring AI:

- ChatModel and ChatClient
- CompositeChatModel (Spring AI Extension presented in **Chapter 7**)
- PromptTemplate
- StructuredOutputConverter

Subtly, using these APIs also involves other, less apparent features and functions of Spring AI, such as the use of `ModelOptions` and Roles to distinguish System and User inputs. Let's cover each component and the prompts in more detail.

Prompts and structuring outputs

In order to understand the Connect4 application's use of `ChatClient` and *Prompt Templates*, we must first review the prompts that were crafted to play Connect4. There were two prompts used in the application.

A System prompt was used to give the AI model specific instructions on how to play Connect4 along with its role.

The System prompt was stated as follows:

You are a player in the two-player board game Connect4. The game board is a two-dimensional grid with six rows and seven columns. A player chooses a column and plays either the letter "X" or the letter "O". To win, a player must be the first one to play the same letter in four adjacent cells of the grid, either in the same row, the same column, or diagonally. Play continues until a player wins or there are no more available moves.

To build on the System prompt used to ground the AI model, a User prompt is then used to request that an AI model play a move based on the current state of the game board. The User prompt was also used to cite the AI model's directive, that you should win when possible and prevent your opponent from winning.

The User prompt was stated as follows:

You are playing letter "{playerDisc}". The current state of the game board is "{gameBoard}". Think carefully and strategically about your next move. Minimize the number of moves needed to connect 4 and beat your opponent. If you have an opportunity to win, then you must make a move to win. If your opponent has an opportunity to win, then you should block your opponent and prevent them from winning. Select 1 of the available columns represented as a letter in {availableColumns}. What is your move? Explain.

As seen in the User prompt above, placeholders, such as `{playerDisc}`, were used to inform the AI model about the disc it will be playing, either the letter "X" or the letter "O." The AI models were also presented with the current state of the game board.

> TIP
>
> While the prompts were hardcoded as `Strings` in the Connect4 application for simplicity's sake, prompts can be sourced from a Spring `Resource` using the `ChatClient` API. This can be done using `chatClient.prompt().user(promptUserSpec -> promptUserSpec.text(:Resource)).call()`. As a result, prompts can then be sourced from anywhere, such as from files, a data store, a prompt registry, SCM, etc. This would further allow prompts to be properly managed and maintained.

Prompts aren't the only thing that is important. The data structure used to present game state to an AI is equally important as we will learn next.

The game board data structure

The representation of the game board turned out to be a key component in getting the AI models to play somewhat intelligently. If experience teaches you anything, it will be that data structures make all the difference when implementing an algorithm. Using the right data structure can simplify the algorithm as well as make it more optimal.

AI is no exception. Choosing an appropriate representation or data structure for the game board that is presented to the AI model is key to better results. An appropriate data structure is the difference between an AI model making an intelligent move vs. a bonehead one, such as playing an unavailable cell, like F7 when there are only 6 rows.

While you saw a few examples of user-friendly representation of the game board above, such as:

```
 A  B  C  D  E  F  G
 --- --- --- --- --- --- ---
|  |  |  |  |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  |  |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  |  | O |  |  |
 --- --- --- --- --- --- ---
|  |  |  |  | O |  |  |
 --- --- --- --- --- --- ---
|  |  |  | X | O |  |  |
 --- --- --- --- --- --- ---
|  |  | X | X | O |  |  |
 --- --- --- --- --- --- ---
```

The AI model actually sees the following:

```
-------
-------
----O--
----O--
---XO--
--XXO--
```

The Xs and Os represent the current plays and the dashes (-) represent no play. This is clearly a two-dimensional array.

> Not only is this two-dimensional array a simple representation of the game state, but it is also an optimal data structure for reducing the token count, thereby reducing costs. The representation only uses 15 tokens as determined by OpenAI's Tokenizer. Copy the textual representation above into https://platform.openai.com/tokenizer.

Lastly, the AI model was simply given a choice of available columns to play, asked to make a move, and then explain. Obviously, an AI model cannot play a column that is completely filled, or at least it shouldn't. So, if a column is unavailable, it's not an option.

Both prompts are sent to the AI model on each request. Arguments for the prompt template were supplied to the `ChatClient` when prompting the AI model. The arguments to the prompt (template) are constructed as follows:

```
Map<String, Object> promptTemplateArguments = Map.of(
  "gameBoard", "\n\n%s\n\n".formatted(boardGame.getGameBoardStateAsGrid()),
  "playerDisc", player.disc().getSymbol(),
  "availableColumns", Arrays.toString(boardGame.getPlayableColumnsAsLetter())
);
```

An example of the "gameBoard" is shown above. The "playerDisc" is either "X" or "O" and available column letters appear as "[A, B, C, E, F, G]". The letter "D" is not shown when the column is not available to play or is full.

The prompt template arguments get passed to the `promptModel(..)` method to prompt the AI model using the configured `ChatClient`, as shown below:

```
Play promptModel(String model, Map<String, Object> promptTemplateArguments, ChatClient chatClient) {

  ...  return chatClient.prompt()
    .system(systemPromptTemplate())
    .user(promptUserSpec -> promptUserSpec
      .text(userPromptTemplate())
      .params(promptTemplateArguments))
    .options(Utils.buildChatOptions(model))
    .call()
    .entity(jsonConverter);
}
```

As seen in the `promptModel(..)` method above, both System, and User prompt templates are supplied to the `ChatClient`. The prompt arguments are used to populate placeholders in the User prompt. The `ChatClient` returns the JSON response generated by the AI model formatted as a "structured" `Play` domain model object. The AI model is dynamically configured at runtime using model options.

The `promptModel(..)` method from the `AbstractConnectFourApplication` class was explained earlier in this chapter.

The `Play` type is a Java record used to capture the move and explanation made by the AI model. The definition of the `Play` record appears as follows:

```
public record Play(String move, String explanation) { ... }
```

In order to inform the AI models to return a response in JSON, this required the use of Spring AI's `BeanOutputConverter`. The `BeanOutputConverter` creates a JSON Schema using the `Play` type defined above. The JSON Schema augments the complete prompt sent to the AI model as part of the request.

The `BeanOutputConverter` is also responsible for converting the JSON into an instance of `Play`, populated with both the move and explanation sourced from the JSON returned in the AI model's generated response.

However, using the `BeanOutputConverter` was not sufficient by itself. Certain AI models require further instructions by using `ChatModel` options to precisely instruct the AI model to generate and return JSON based on the given JSON schema.

A JSON schema only informs the AI model about the structure of the JSON. It does not imply nor guarantee that the AI model will respond with JSON.

Both MistralAI and Ollama need to be specifically configured to return JSON by declaring the following properties in `application.properties`, as seen here:

```
spring.ai.mistralai.chat.options.responseFormat=json_object
spring.ai.ollama.chat.options.format=json
```

Both AI models needed this additional configuration in `application.properties` despite explicit instructions given by the `getFormat()` method in `BeanOutputConverter`, which states:

"*Your response should be in JSON format*."

Do not include any explanations, only provide a RFC8259 compliant JSON response following this format without deviation."}

Do not include markdown code blocks in your response.

Remove the ```json markdown from the output.

Here is the JSON Schema instance your output must adhere to:

```%s```"

The instructions are automatically supplied when the `BeanOutputConverter` is provided to the `entity(..)` builder method on `ChatClient`.

Neither Google Gemini nor OpenAI specifically needed additional model options instructing the AI to format the response as JSON.

Even after configuring a `BeanOutputConverter` and setting model options using Spring AI properties, Ollama still needed further tuning since certain AI models would occasionally return malformed JSON, such as JSON missing a closing bracket '}.'

Additionally, heeding Spring AI's advice to sanitize the JSON returned in the AI model's generated response, as documented (https://docs.spring.io/spring-ai/reference/api/structured-output-converter.html; see tips), I created an additional Spring AI Extension to sanitize the JSON using Google's `JsonSanitizer` class.

> TIP
>
> See more details from OWASP here: https://owasp.org/json-sanitizer/. Sanitizing JSON is a good practice in any case, especially when it comes to security. Inputs should always be validated!
```

To validate the JSON, I created another Spring AI Extension, the `JsonSanitizerOutputConverter` class, which wraps an existing `StructuredOutputConverter`. `JsonSanitizerOutputConverter` first sanitizes the JSON generated by the AI model using the `JsonSanitizer` class, then delegates to the wrapped `StructuredOutputConverter` to convert the JSON to a `Play` object.

The `JsonSanitizerOutputConverter` is loosely implemented as follows:

```
import com.google.json.JsonSanitizer;
import o.s.ai.converter.StructuredOutputConverter;

public class JsonSanitizerOutputConverter<T>
    implements StructuredOutputConverter<T> {

  private final StructuredOutputConverter<T> delegate;

  @Override
  public String getFormat() {
    return this.delegate.getFormat();
  }

  @Override
  public @NonNull T convert(@NonNull String json) {
    String sanitizedJson = JsonSanitizer.sanitize(json);
    return this.delegate.convert(sanitizedJson);
  }
}
```

As seen in the converter class above, the JSON is sanitized before it is converted.

After the configuration was set up properly, the AI models consistently returned well-formed JSON that could be converted into a `Play` object. The `Play` object was then further wrapped in a `PlayerAction` to associate the `Play` made by the AI model with the AI model itself, as shown above and in the following code snippet:

```
PlayerAction playerAction = PlayerAction.by(currentPlayer)
    .played(play).in(milliseconds);
```

The `PlayerAction` is defined as:

```
public record PlayerAction(Player player, Play play, Duration time) { ... }
```

The Java record captures the `Player` (AI model), the `Play` (move + explanation) and the length of time in milliseconds for the AI model to make a move, which you can see in the output:

```
2025-10-26T22:41:34.700-07:00  INFO 85744 --- [Connect4 App] [           main] c.ConnectFourApplication$$SpringCGLIB$$0 : AI model decision duratio
```

Now, let's discuss a bit on the AI models' explanations and use of reasoning models to determine if there is any sound logic to an AI model's moves at all.

Explanations and AI model reasoning

Let's face it, all of the AI models used in the Connect4 application won't outsmart a human anytime soon. Despite their seemingly and sometimes reasonable explanations, most, if not all, AI models made suboptimal moves as well. Of course, in AI's defense, this can be attributed to several factors.

First, prompt engineering is everything, as is data modeling. Creating an effective prompt is not a simple one and done task. It takes many, many iterations of fine tuning to craft a prompt that obtains the desired results. This is further exacerbated by the fact that AI models are constantly changing, dare I say evolving. Plus, not all AI models were made equal.

Even after you fine-tune a prompt, the moment you upgrade to a newer version of the same AI model, or especially when changing AI models or providers, you most certainly will need to tune your prompts again and again.

The power of an AI model depends on training data, parameter size, and algorithms applied in its implementation. As mentioned in other chapters, data is everything, as is the number of parameters an AI model uses to probabilistically pull from when making an evaluation and generating its response.

I also did not tune any other model options. My intent was to use a model out-of-the-box, as is, without much customization or fine-tuning. However, I did choose from a variety of AI models, such as more advanced reasoning models, to determine if certain AI models have an advantage over others.

Locally, I was constrained by the limited resources available on my computer, and I couldn't run AI models much larger than ~15 GB in size, with ~8 billion parameters. Therefore, it is unclear whether larger parameter models trained on more data have a distinct advantage over smaller parameter models.

Using OpenAI or Google AI models in the cloud boils down to cost, especially when you start using "reasoning" models. See Google's (https://cloud.google.com/vertex-ai/generative-ai/pricing#gemini-models-2.5) and OpenAI's (https://platform.openai.com/docs/pricing) price sheets for more details.

In any case, "reasoning" models didn't fare much better in competition. I guess the pressure got to them. Here is an example of seemingly sound reasoning, but a complete bonehead move:

```
Current player is [OpenAI] playing [X]
AI model explanation [Play in column D. The X will drop to row 3 (above the two existing Xs at rows 4–5), creating three Xs in a vertical line (rows 3–5) an
AI model decision duration 50767 ms
 A  B  C  D  E  F  G
 --- --- --- --- --- --- ---
|  |  |  |  |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  |  |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  | X |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  | X |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  | X | O |  |  |
 --- --- --- --- --- --- ---
|  |  |  | O | O |  |  |
 --- --- --- --- --- --- ---
Press <enter> to continue next play
```

Well played, OpenAI! Now, clearly Ollama with Meta's llama3.2 needs to be careful here. Let's see:

```
Current player is [Ollama] playing [O]
AI model explanation [Placing an O in column F blocks X's potential four-in-a-row and creates a chance for a future win]
AI model decision duration 1769 ms
 A  B  C  D  E  F  G
 --- --- --- --- --- --- ---
|  |  |  |  |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  |  |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  | X |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  | X |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  | X | O |  |  |
 --- --- --- --- --- --- ---
|  |  |  | O | O | O |  |
 --- --- --- --- --- --- ---
Press <enter> to continue next play
```

"Placing an O in column F blocks X's potential four-in-a-row and creates a chance for a future win." While it nicely set up a guaranteed win (if only llama3.2 had another move), clearly, Meta's llama3.2 should have thought a bit "longer" and harder about its next move. And, no surprise here, OpenAI for the win:

```
Current player is [OpenAI] playing [X]
AI model explanation [Play in column D. There are three X pieces stacked in that column; dropping an X into D will land on top of them and create four X
AI model decision duration 22096 ms
 A  B  C  D  E  F  G
 --- --- --- --- --- --- ---
|  |  |  |  |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  | X |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  | X |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  | X |  |  |  |
 --- --- --- --- --- --- ---
|  |  |  | X | O |  |  |
 --- --- --- --- --- --- ---
|  |  |  | O | O | O |  |
 --- --- --- --- --- --- ---

[OpenAI] playing [X] wins!
Winner winner chicken dinner!
```

In other games, despite giving AI models specific instructions and the current state of the game board along with available moves, some AI models still selected columns that were full, leading to a `RuntimeException`. \ A sensible handler might be to have the AI model retry its move. However, instead I opted to have the application log the available columns, state of the game board and the AI model's selection, then exit the program. This was evident in the program (**REPL**) loop, as seen calling the `playSafely(..)`, for example:

```
playSafely(gameBoard -> {
  gameBoard.play(playerAction);
  gameBoard.printGameBoard();
}).accept(boardGame);
```

The `playSafely(..)` method was implemented as follows:

```
return boardGame;
   try {
      boardGameConsumer.accept(boardGame);
   }
   catch (RuntimeException cause) {
      logWarn("Available Columns {}",
         Arrays.toString(boardGame.getPlayableColumnsAsLetter()));
      logWarn("Connect4 Game Board State [{}]",
         boardGame.getGameBoardStateAsGrid());
      throw ConnectFourException
         .because("AI model fumbled the ball", cause);
   }
};
```

You should be careful about retries, especially time-based or unlimited tries. Each failed attempt will cost additional tokens.

For all my prompt refinements and testing, I played around with the following AI models:

- MistralAI's `magistral-small-latest` model running in the cloud
- OpenAI's `gpt-oss:20b` model running with Ollama
- Meta's `llama3.2` model running with Ollama
- DeepSeek's `deepseek-r1:8B` model running with Ollama
- OpenAI's `gpt-5-mini` model running in the cloud
- Google's `gemini-2.5-flash-lite` model running in the cloud

While results were mixed, OpenAI GPT 5 consistently and decisively won.

In the end, I really think further prompt tuning and specialized training, thereby teaching an AI model the intricacies of playing Connect 4 with proven strategies, is the most optimal approach here. I cannot overemphasize the importance of prompt tuning and data modeling. It is essential every single time.

Summary

In this chapter we saw how Spring AI can be used to interact with multiple AI models, pitting them against each other in a game of Connect4.

We composed OpenAI, Ollama, MistralAI and Google Vertex AI Gemini into a single `ChatModel` implementation and interacted with the AI models using a `ChatClient`. The `ChatClient` was given both System and User prompts, and the User prompt used a *prompt template* accepting arguments at runtime in order to inform the AI model about the state of game play.

The AI models generated a response in JSON that was sanitized and converted into a domain object using `StructuredOutputConverters` for easier processing.

After all was constructed, it becomes a simple matter to switch AI models simply by changing a property. Of course, adding an additional AI provider still requires another Spring AI Model Starter declaration in the Maven POM, but it is fairly simple to set up and run experiments.

Along the way, we learned the importance of the data structures used to model information on which the AI model makes decisions. When combined with carefully crafted and contextually relevant prompts, the results can lead to better outcomes. However, the parameter size and reasoning models made little difference in the performance exhibited after several trials despite their occasional and seemingly sound reasoning. OpenAI GPT-5 exhibited better performance and "reasoning" than its peers, though I imagine this will quickly change as models continue to evolve. After all, no one likes to lose, even the AI providers.

I think a more interesting experiment would be to specifically train an AI model on how to effectively play Connect4 on par with playing a human. While I am left feeling a bit disappointed by AI's performance, it was a fun little exercise, nevertheless.

In the next chapter we dive into advanced features of Spring AI in order to build robust, intelligent applications.

Subscribe for a free eBook

New frameworks, evolving architectures, research drops, production breakdowns – *AI_Distilled* filters the noise into a weekly briefing for engineers and researchers working hands-on with LLMs and generative AI systems. Subscribe now and receive a free eBook, along with weekly insights that help you stay focused and informed.

Subscribe at https://packt.link/8Oz6Y or scan the QR code below.



Previous chapter

‹ Chapter 5: Beat Shazam

Next chapter

Part 3: Beyond Simple Intelligence ›